

— IoT Devices

Les 1 - Introductie



IoT Devices, hmm...? Dat ken ik van ergens!

- Deze module **gaat verder** op de opgedane kennis uit het vak Microcontrollers (in fase 1).
- Wat hebben we daar geleerd?
 - Werken met een Nucleo microcontrollerbord: NUCLEO-091RC.
 - Programmeren in C-taal.
 - Basis input/output verwerken.
 - Communiceren via UART, SPI en I²C.
 - Gebruik maken van de **CMSIS-bibliotheken**.

Doelstellingen voor **deze module**?

- Leren werken met de NUCLEO-**L476RG**.
 - Nog steeds toegang tot UART, SPI, I²C, CAN, timers, AD, DA, ...
 - Maar nu ook USB-interface (CDC), I²S-interface, ...
 - Kloksnelheden tot **80MHz**.
- Leren werken met de **HAL-bibliotheken**.
 - Vorige module vooral gewerkt op registerniveau met CMSIS-code.
 - **Nu Hardware Abstraction Layer bibliotheek gebruiken.**

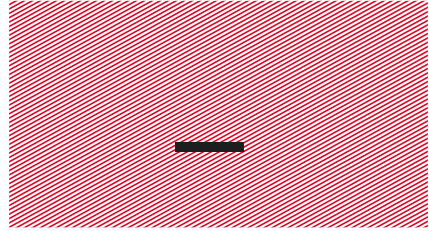
Doelstellingen voor deze module?

- De verschillende soorten IO-transfers inoefenen:
 - polling.
 - interrupts.
 - **Direct Memory Access (DMA)**.
- Beveiligde **WiFi-communicatie** opzetten met een ESP32 C3-module via AT-commando's.
- **Bluetooth communicatie** opzetten naar een webbrowser.
- Leren werken met een **Realtime Operating System (RTOS)** op een microcontroller.
- Dieper ingaan op correct gebruik van de **C-taal**.



Hoe?

- **Theorie**
 - Presentaties
 - Uitleg
 - Live demo's
 - Theorieopdrachten
- **Labo**
 - Zelfstandig oefenen
 - Labo-opdrachten
 - (Mondelinge) evaluatie

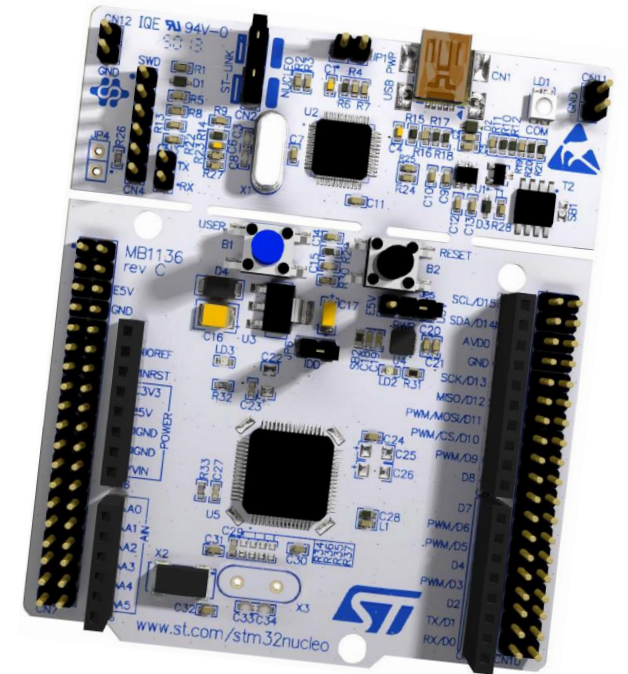


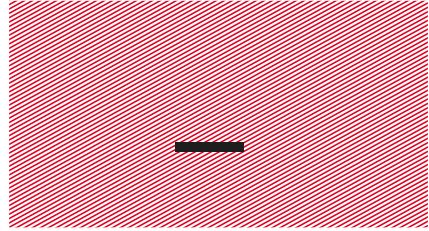
NUCLEO-L476RG



NUCLEO-L476RG

- Ziet er hetzelfde uit als de andere Nucleo 64 borden.
- Kan dus ook uitgebreid worden met het Nucleo Extension Shield (knoppen, LED's, AD en 10-pin connector).
- Bezit de STM32L476RG microcontroller.
- Heeft ook een ST-link programmer on board.
- Kan geprogrammeerd worden via Keil μ Vision met behulp van C-taal.
- Wordt ondersteund door STM32CubeMX en de HAL-bibliotheken.





CMSIS versus HAL



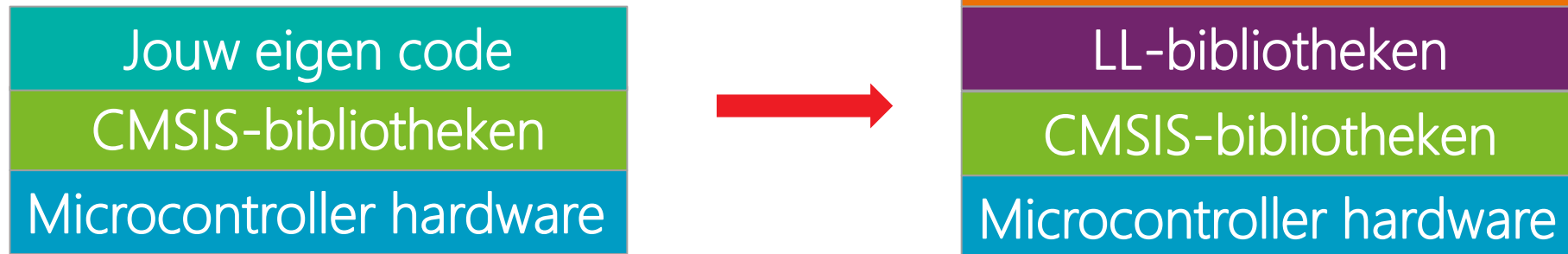
- In het verleden werkten we met CMSIS-code. Die manier van werken is **heel goed**, omdat er een standaard manier van werken gehanteerd wordt: Cortex Microcontroller Software Interface Standard.
- Je kan er heel efficiënt tot **op bit-niveau in de registers** zaken gaan aanpassen.

```
// Clock voor GPIOA inschakelen.  
RCC->AHB2ENR = RCC->AHB2ENR | RCC_AHB2ENR_GPIOAEN;
```

- **Echter**, veranderen van type microcontroller is niet zo eenvoudig...

CMSIS versus HAL

- Indien we vlotter van microcontroller willen veranderen, **kan** een Hardware Abstraction Layer (HAL) een voordeel bieden.
- De HAL-bibliotheek is een softwarelaag (C-taal) die **bovenop** de CMSIS-bibliotheken ligt, waardoor je onafhankelijk(er) wordt van het type microcontroller...



- **Mogelijks gevaar:** het maakt code soms wel minder efficiënt...

Codevoorbeeld voor het omkappen van de toestand van een LED.

- CMSIS

```
GPIOC->ODR = GPIOC->ODR ^ GPIO_ODR_OD0;
```

- HAL

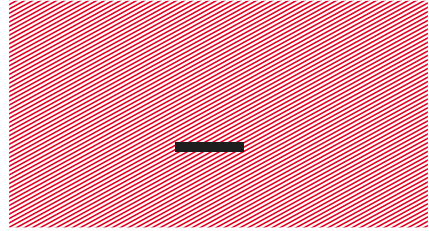
```
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
    uint32_t odr;

    /* Check the parameters */
    assert_param(IS_GPIO_PIN(GPIO_Pin));

    /* get current Output Data Register value */
    odr = GPIOx->ODR;

    /* Set selected pins that were at low level, and reset ones that were high */
    GPIOx->BSRR = ((odr & GPIO_Pin) << GPIO_NUMBER) | (~odr & GPIO_Pin);
}
```

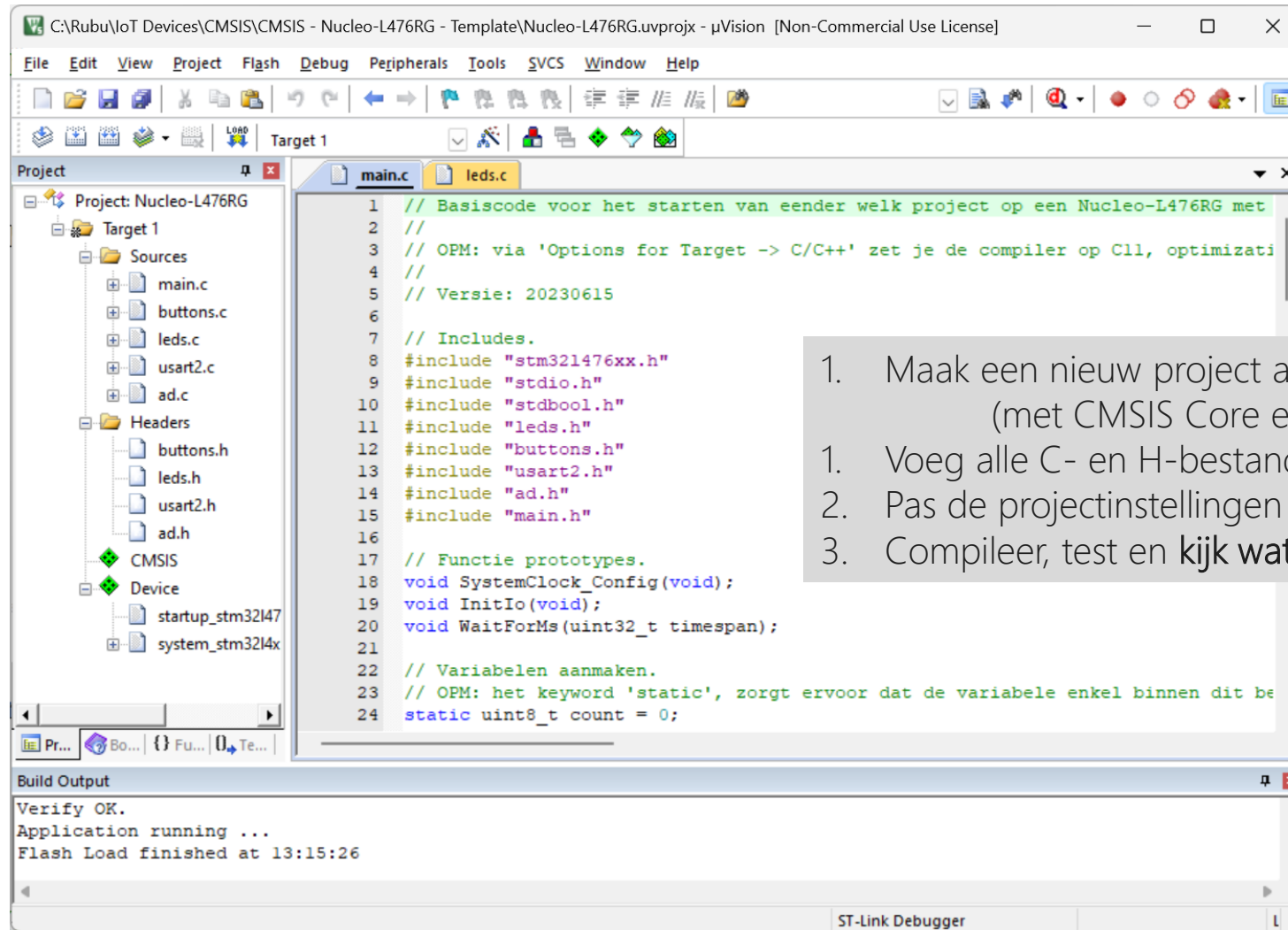
Later bekijken we in detail wat hier gebeurt, maar nu zie je al duidelijk dat er meer code nodig is voor hetzelfde resultaat...



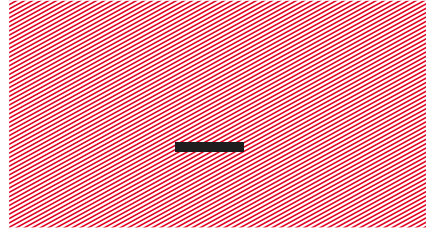
Demo: CMSIS template in Keil μ Vision

- Doel?
 - Het **basiscodevoorbeeld** om het Nucleo Extension Shield te testen, programmeren in een NUCLEO-L476RG.
- Hoe?
 - Gebruik Keil μ Vision om 'vanaf nul' een project op te bouwen.
 - Bekijk de reference manual en de device specifieke **datasheet** om de nodige registerinformatie op te zoeken.
- Vaststelling?
 - De opgedane **kennis** van in de module Microcontrollers, **kan je rechtstreeks gebruiken** om hier tot een goed resultaat te komen.

CMSIS template



1. Maak een nieuw project aan in µVision voor de STM32L476RG chip (met CMSIS Core en startup).
1. Voeg alle C- en H-bestanden toe (beschikbaar op Toledo).
2. Pas de projectinstellingen aan (ST-link, compiler, ...).
3. Compileer, test en kijk wat rond (in het project)...



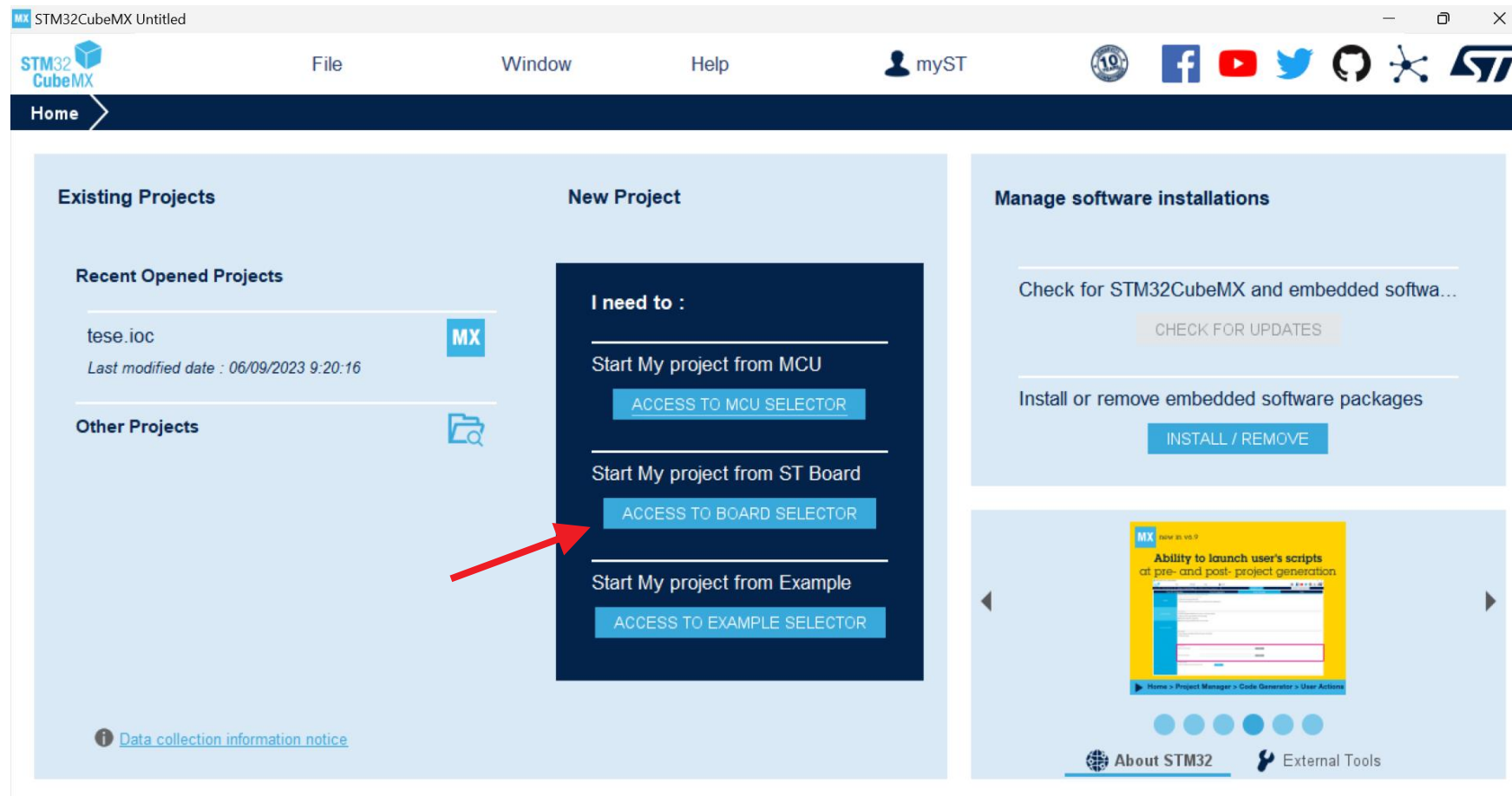
Demo: HAL template met STM32CubeMX en Keil μ Vision



- Doel?
 - **Hetzelfde** basiscodevoorbeeld om het Nucleo Extension Shield te testen, programmeren in een NUCLEO-L476RG.
- Hoe?
 - Gebruik **STM32CubeMX** om een Keil μ Vision project aan te maken.
 - Zet daarbij de juiste instellingen om bijvoorbeeld de LED's en knoppen als uitgang en ingang te definiëren.
- Vaststelling?
 - We kunnen dus op **een 'grafische' manier**, ook de microcontroller instellen...

HAL template

Start STM32CubeMX op en kies 'Start My project from ST Board'.



HAL template

Selecteer onze Nucleo-versie en klik op 'Start Project'.

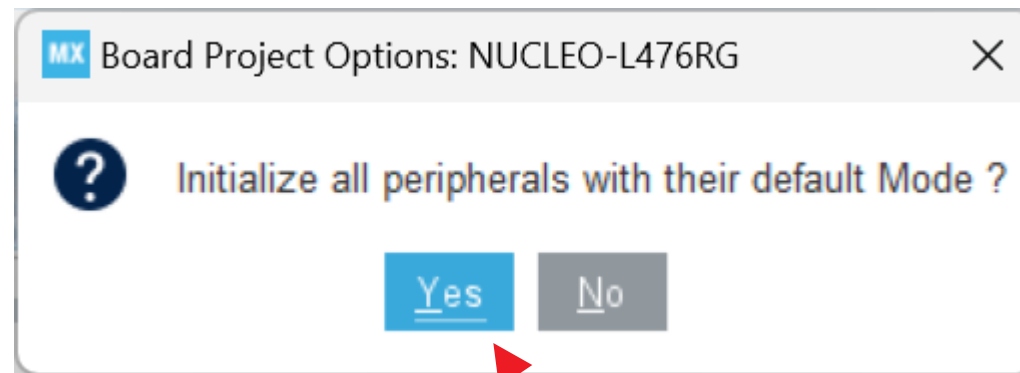
The screenshot shows the 'New Project from a Board' web interface. The 'Board Selector' tab is active. On the left, the 'Board Filters' section shows 'Commercial Part Num' set to 'NUCLEO-L476RG'. Below this, 'PRODUCT INFO' and 'MEMORY' sections are visible. The main content area displays the 'STM32L4 Series' with a featured card for 'NUCLEO-L476RG'. This card includes details like 'ACTIVE' status, part number, unit price (\$14.0), and a description of the board. Below the featured card, a 'Boards List' table shows one item: 'NUCLEO-L476RG'. In the top right corner of the interface, the 'Start Project' button is highlighted with a red arrow.

* Overview	Commercial Part No	Type	Marketing Status	Unit Price (US\$)	Mounted Device
	NUCLEO-L476RG	Nucleo-64	Active	14.0	STM32L476RGT3



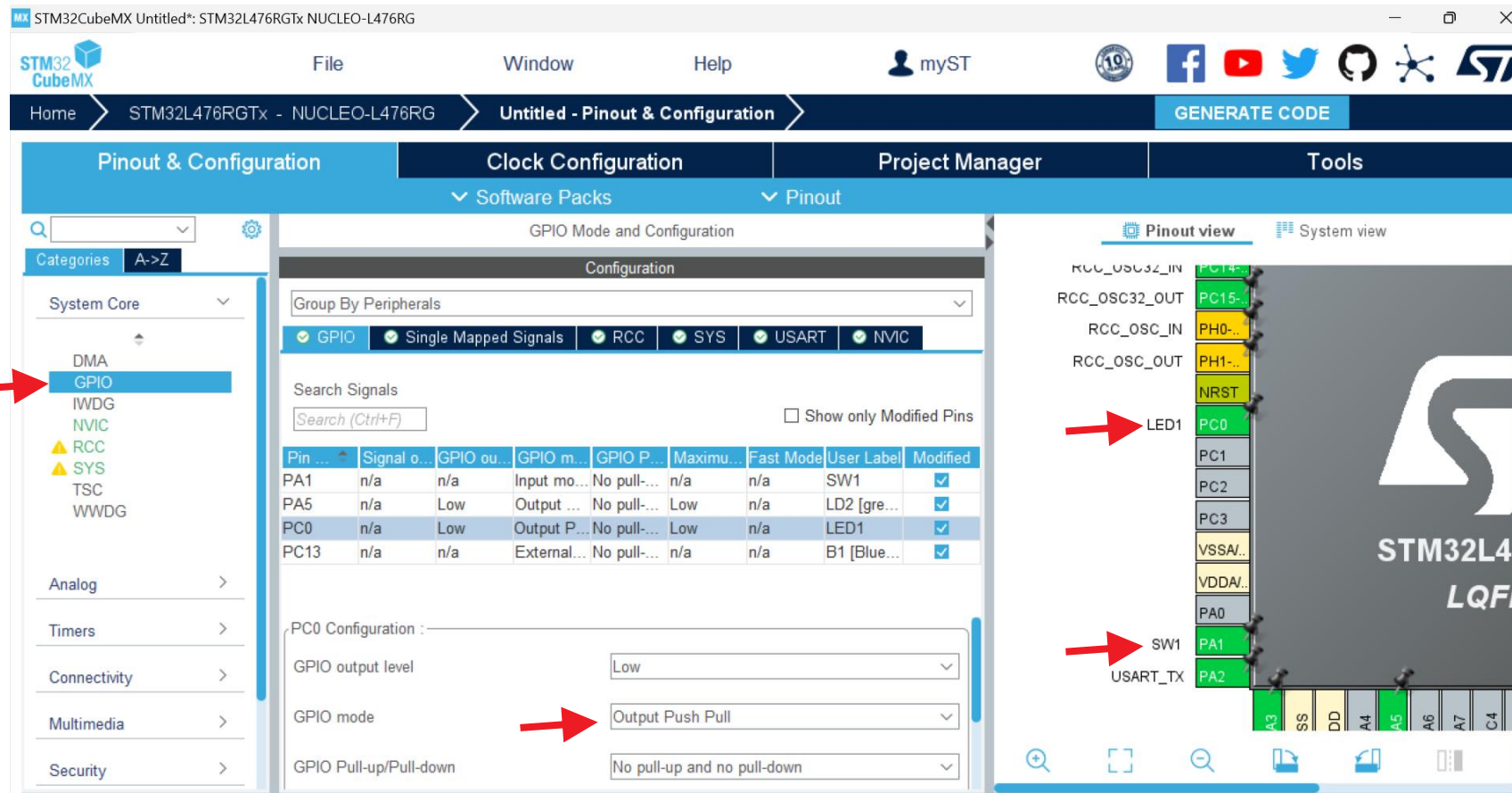
HAL template

Initialiseer de randapparatuur met de standaardinstellingen.



HAL template

Zoek op in het schema van het Nucleo Extension Shield, waar LED1 en SW1 mee verbonden zijn. Stel die twee IO's in zoals bedoeld...



HAL template

Ga naar het tabblad 'Project Manager' en bepaal daar de projectnaam, -locatie en de IDE waarmee je wil werken (µVision = MDK-ARM).

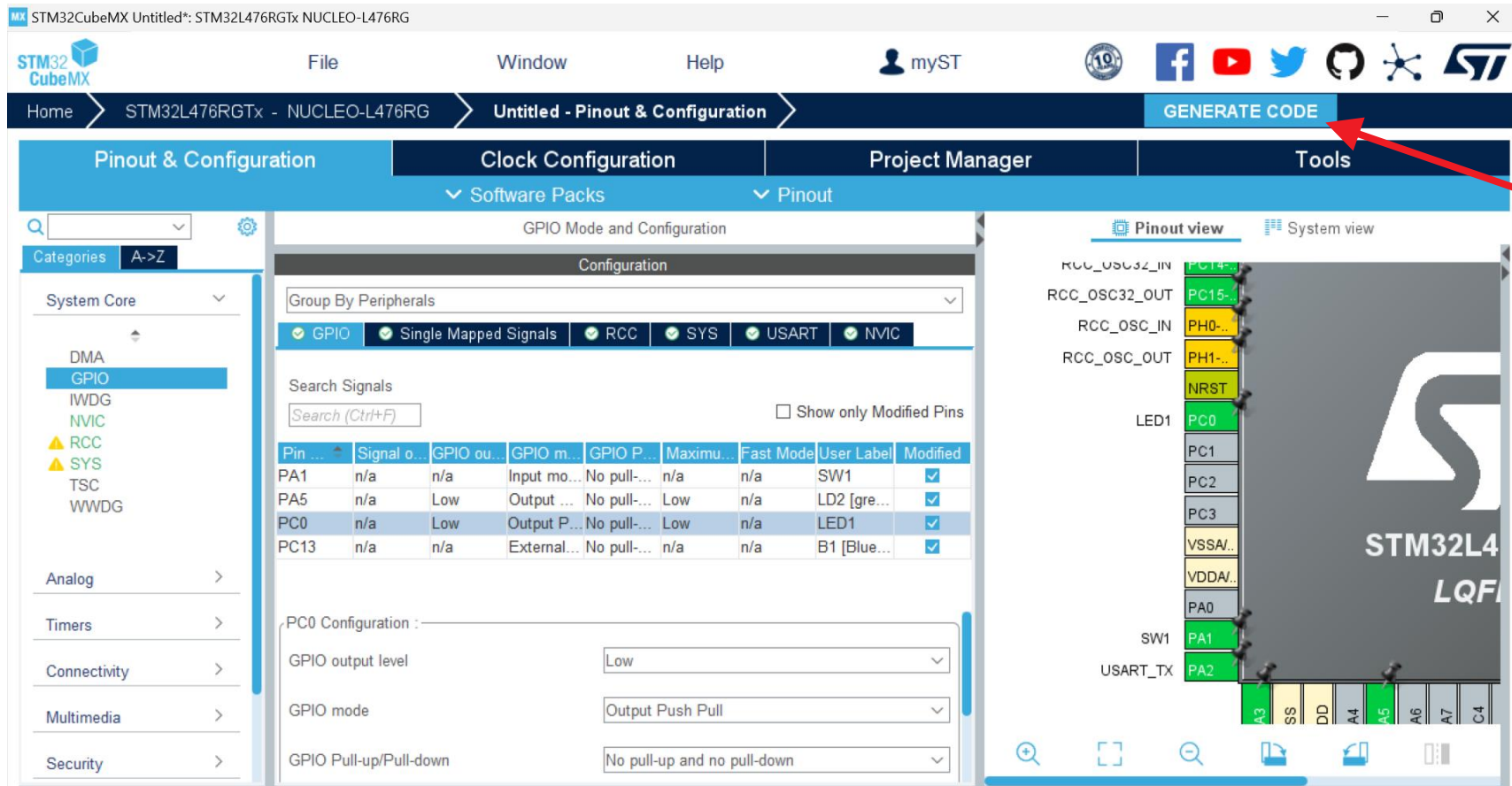
The screenshot shows the STM32CubeMX Project Manager window. The 'Project Manager' tab is selected. The left sidebar has four sections: 'Project', 'Code Generator', 'Advanced Settings', and 'Tools'. The 'Project' section is expanded, showing the following settings:

- Project Name: MX Test
- Project Location: C:\temp (with a 'Browse' button)
- Application Structure: Advanced (dropdown menu)
- Toolchain Folder Location: C:\temp\MX Test\
- Toolchain / IDE: MDK-ARM (dropdown menu)
- Min Version: V5.32 (dropdown menu)
- Linker Settings:
 - Minimum Heap Size: 0x200
 - Minimum Stack Size: 0x400
- Thread-safe Settings:
 - Cortex-M4NS
 - ☐ Enable multi-threaded support
 - Thread-safe Locking Strategy: Default – Mapping suitable strategy depending on RTOS selection. (dropdown menu)
- Mcu and Firmware Package:
 - Mcu Reference: STM32L476RGTx
 - Firmware Package Name and Version: STM32Cube FW_L4 V1.18.0
 - ☒ Use Default Firmware Location

Red arrows point to the 'Project Name', 'Project Location', and 'Toolchain / IDE' fields.

HAL template

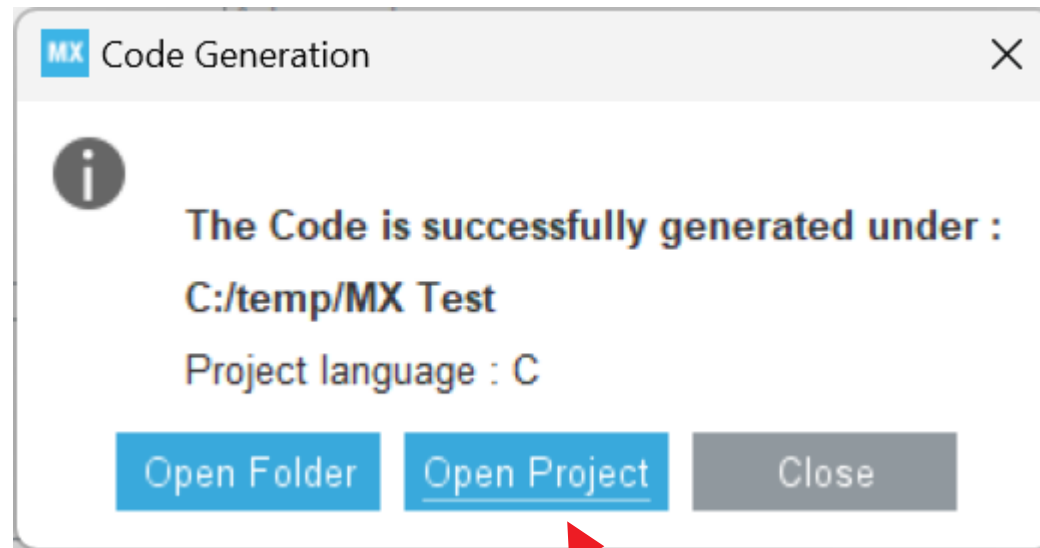
Klik op 'Generate Code' om een Keil μ Vision project aan te maken.





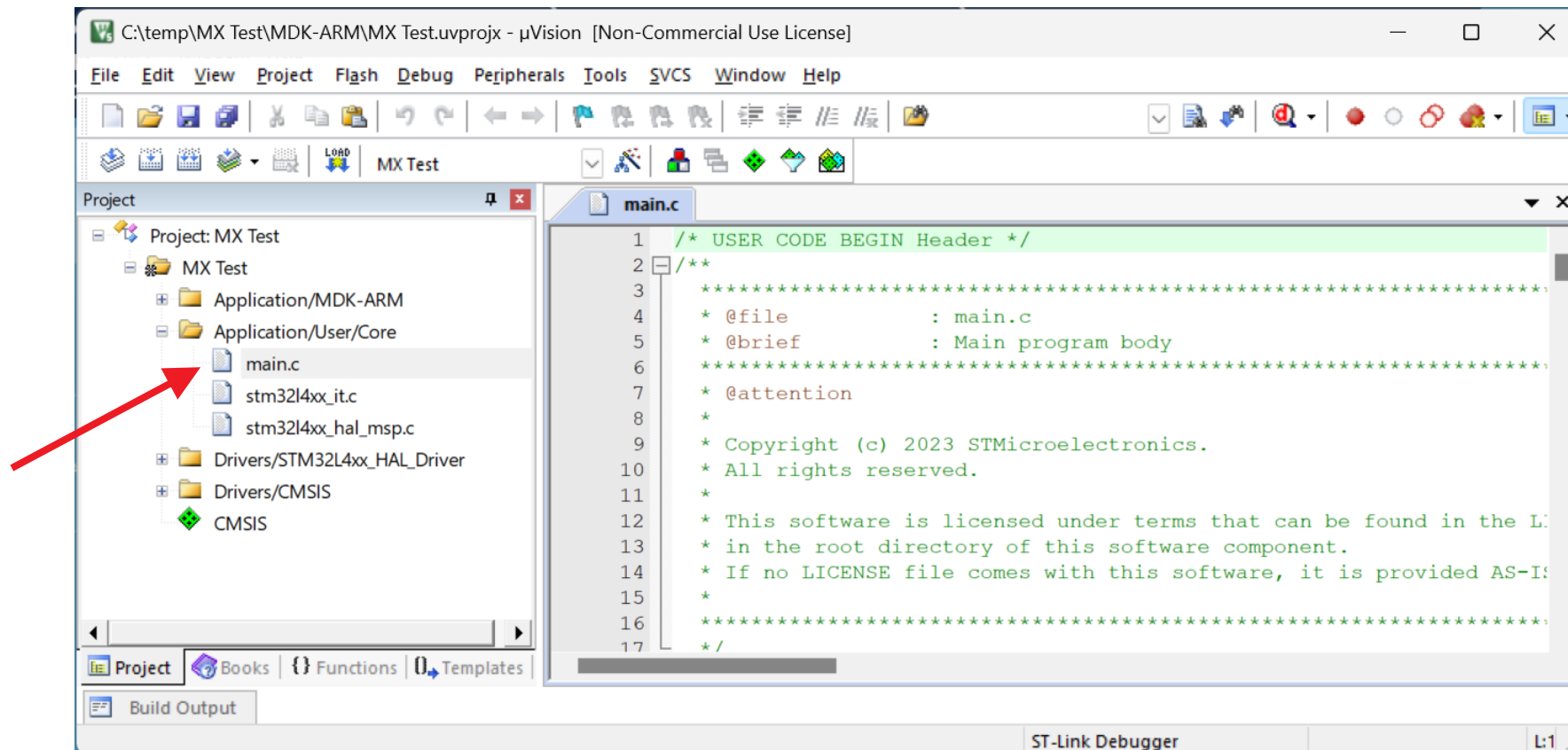
HAL template

Als de code succesvol is aangemaakt, klik dan op 'Open Project'.
Als alles goed gaat, opent nu Keil μ Vision...



HAL template

Het μ Vision project heeft een mappenstructuur zoals hieronder te zien... Open het main.c bestand en snuister eens rond...

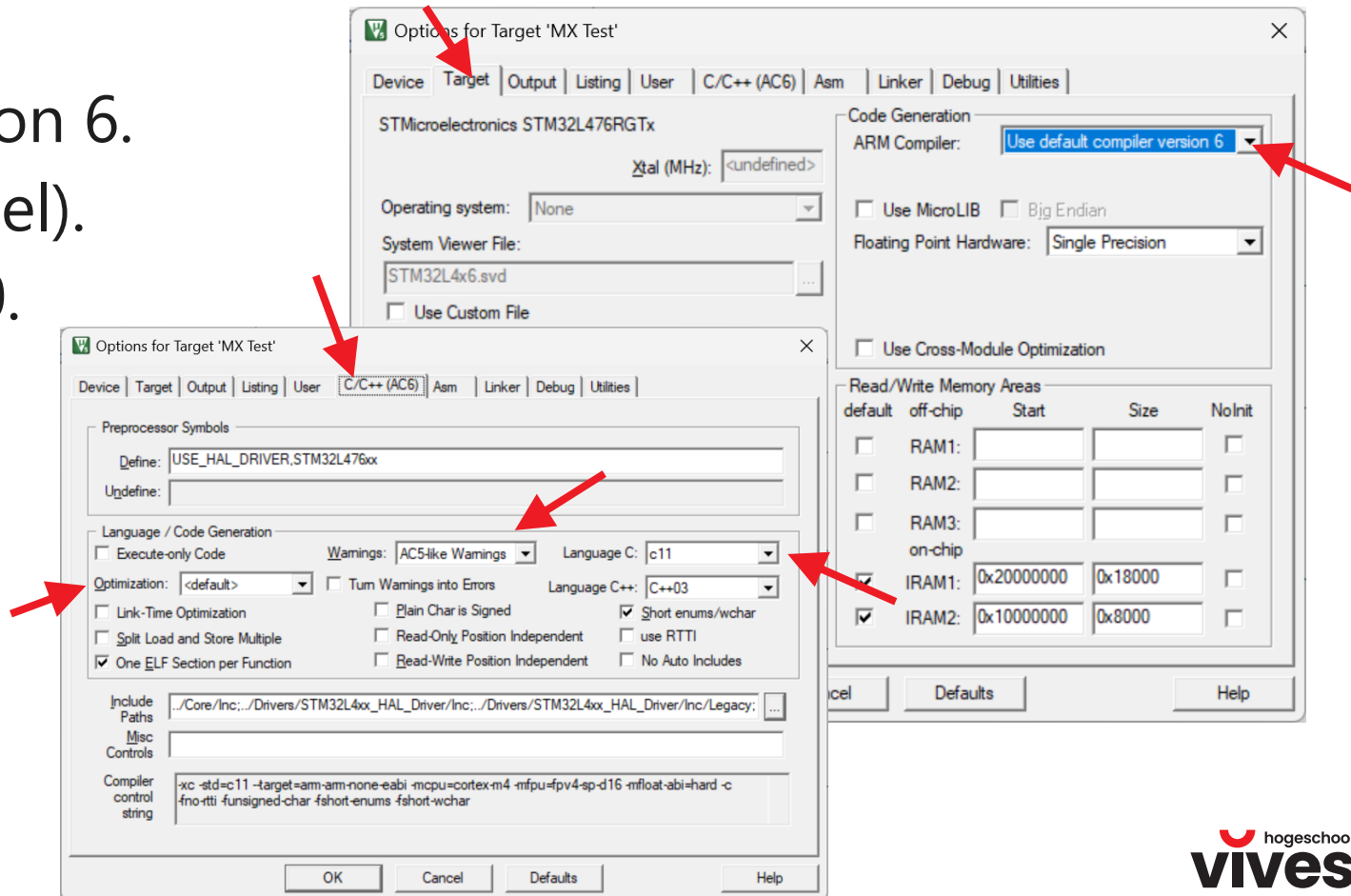


HAL template

Probeer het project te **compileren**. Lukt dat niet, dan moet je wellicht nog enkele instellingen voor de compiler goed zetten.

Denk aan:

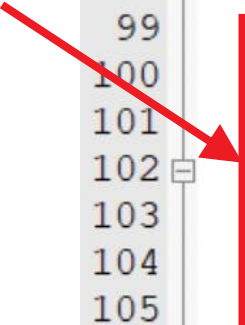
- User default compiler version 6.
- AC5-like warnings (optioneel).
- Optimization: default of O0.
- Language C: C11.
- ST-link debugger kiezen.
- Connect under reset.
- Reset and Run.
- ...



HAL template

De geladen code heeft nu nog geen zichtbare werking. Pas de main.c aan zodat je onderstaande krijgt. **Wat verwacht je?**

```
95  /* Infinite loop */
96  /* USER CODE BEGIN WHILE */
97  while (1)
98  {
99      if (HAL_GPIO_ReadPin(SW1_GPIO_Port, SW1_Pin) == GPIO_PIN_RESET)
100         HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
101     else
102     {
103         HAL_Delay(500);
104         HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);
105     }
106
107     /* USER CODE END WHILE */
108
109     /* USER CODE BEGIN 3 */
110 }
111 /* USER CODE END 3 */
112 }
```



Merk op dat je slechts in bepaalde gedeeltes van de code zelf aanpassingen mag doen. De gedeeltes erbuiten worden automatisch gewist wanneer je STM32CubeMX het project opnieuw laat genereren!!!

Optioneel kan je ook de AD-converter en UART2 instellen.



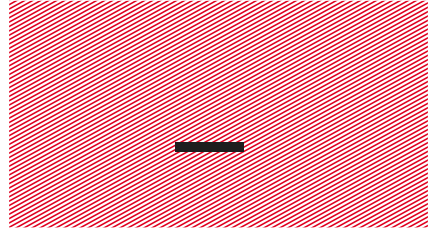


HAL template

Alle pinnen instellen kan een heel werkje zijn, daarom wordt ook een **sjabloon ter beschikking gesteld** via Toledo.

Om de LED's aan te sturen en knoppen uit te lezen, werd ook reeds een 'leds.c' en 'buttons.c' bestand aangemaakt.

Alle bestanden samen vormen het sjabloon. Tijd om dat sjabloon samen te **overlopen**... Bekijk zowel STM32CubeMX als μ Vision.



HAL



- Waar kan ik **info** vinden over de HAL-bibliotheek?
 - Er is een goed uitgewerkte PDF met daarin de essentie...
 - https://www.st.com/resource/en/user_manual/um1884-description-of-stm32l4l4-hal-and-lowlayer-drivers-stmicroelectronics.pdf

**UM1884**

User manual

Description of STM32L4/L4+ HAL and low-layer drivers

- Gebruik een goede PDF-lezer om te navigeren via de bladwijzers. Per 'peripheral' is er een hoofdstuk.

Navigeer in de PDF naar de functie ***HAL_Delay()*** en zoek op wat die doet, welke parameters nodig zijn, wat de return value is,...

HAL_Delay

Function name

```
void HAL_Delay (uint32_t Delay)
```

Function description

This function provides minimum delay (in milliseconds) based on variable incremented.

Parameters

- **Delay:** specifies the delay time length, in milliseconds.

Return values

- **None:**

Notes

- In the default implementation , SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals where uwTick is incremented.
- This function is declared as __weak to be overwritten in case of other implementations in user file.

De HAL-bibliotheek maakt veelvuldig gebruik van **structs** en **pointers** naar structs om de code te structureren ;-).

- **Weet je nog** wat een struct is?
- **Weet je nog** wat een pointer is?
- **Bekijk** de `UART_HandleTypeDef` in een μ Vision project...

Wanneer we de HAL-bibliotheken gebruiken, kan je dikwijls **op verschillende manieren je doel bereiken**.

Je kan bijvoorbeeld de UART Enable bit uit het CR1-register op (minstens) 3 manieren zetten, via:

- `MX_USART2_UART_Init();`
- `__HAL_UART_ENABLE(huart);`
- `huart2.Instance->CR1 |= USART_CR1_UE ;`

Loop door de code om dat uit te zoeken.

Fouten opvangen – `assert_param()`

Je wil uiteraard code schrijven die zo robuust mogelijk is (weinig fouten bevat).

Hier en daar gebruikt men daarvoor de `assert_param()` macro.

Deze macro/preprocessor directive, controleert of de parameters van een bepaald type zijn.

```
void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
{
    uint32_t odr;

    /* Check the parameters */
    assert_param(IS_GPIO_PIN(GPIO_Pin));

    /* get current Output Data Register value */
    odr = GPIOx->ODR;

    /* Set selected pins that were at low level, and reset ones that were high */
    GPIOx->BSRR = ((odr & GPIO_Pin) << GPIO_NUMBER) | (~odr & GPIO_Pin);
}
```

MAAR: zolang de `USE_FULL_ASSERT` niet gedefinieerd wordt in het 'stm32l4xx_hal_conf.h' bestand, heeft de macro geen echte functie...!

Voorlopig niet van belang dus...

Fouten opvangen - Error_Handler()

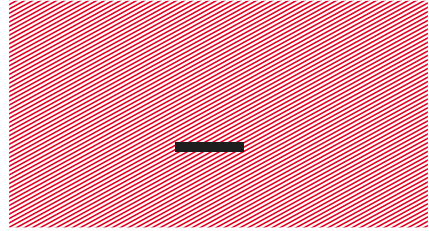
Een andere manier om fouten te ontdekken is het gebruik van de functie *Error_Handler()* in de HAL-bibliotheken.

Als een HAL-actie terugkeert met **iets anders dan *HAL_OK***, wordt die functie (soms) opgeroepen...

```
352  * @brief This function is executed in case of error occurrence.
353  * @retval None
354  */
355 void Error_Handler(void)
356 {
357     /* USER CODE BEGIN Error_Handler_Debug */
358     /* User can add his own implementation to report the HAL error return state */
359     __disable_irq();
360     while (1)
361     {
362     }
363     /* USER CODE END Error_Handler_Debug */
364 }
```



MAAR: ook hier kan je zelf nuttig acties koppelen aan die foutmelding. Op dit moment schakelen we de interrupts uit en blijven we in een oneindig lange lus...!



C-taal



C-taal – C-string

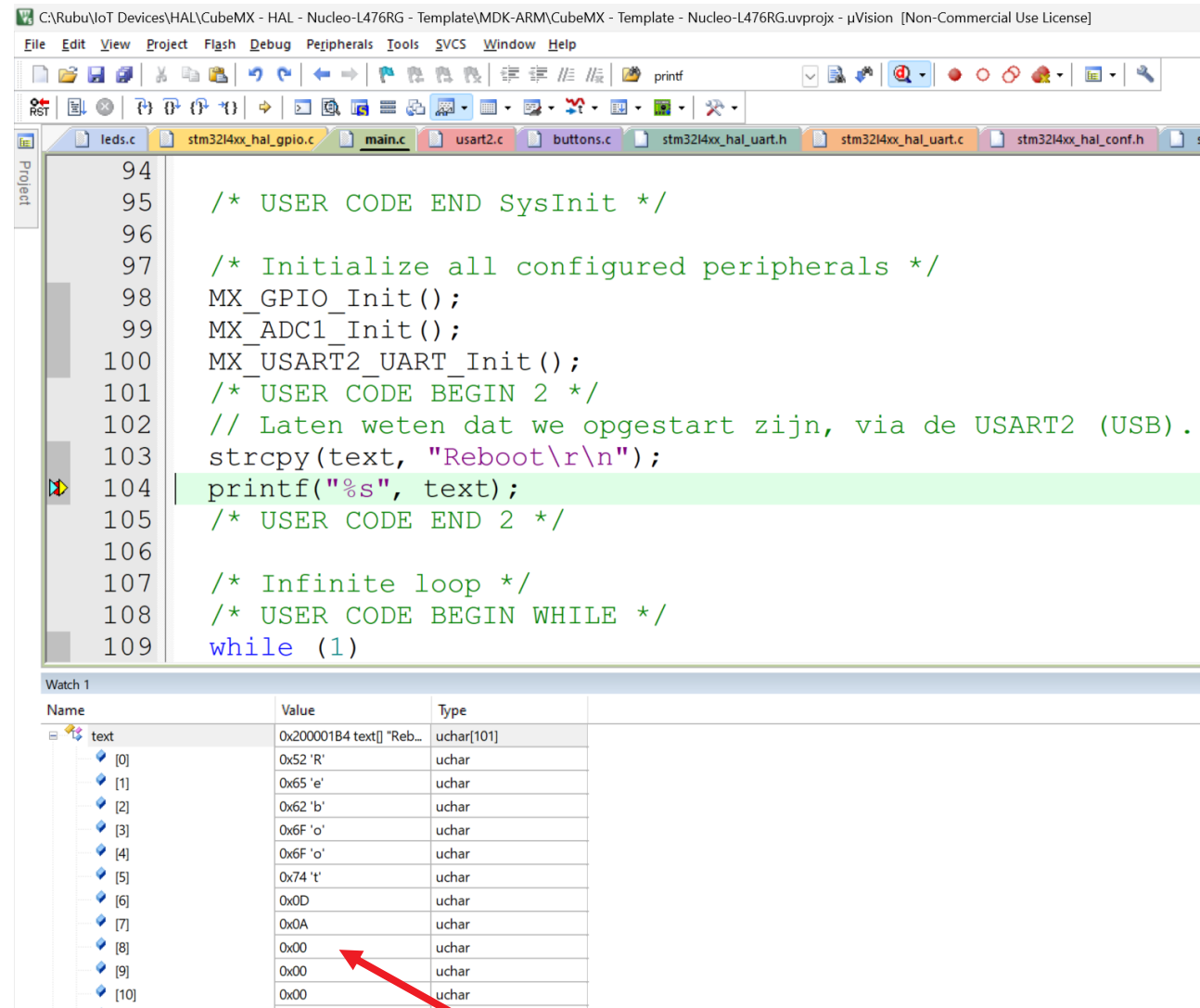
- Een '**tekst**' of 'string' in C-taal wordt bewaard in een array van karakters.
- Om aan te geven dat de tekst stopt, wordt er **op het einde een 'nulkarakter' toegevoegd** ('\0').

V	I	V	E	S	\0
---	---	---	---	---	----

- Je kan dat nulkarakter gebruiken om bijvoorbeeld te **stoppen** met verzenden van karakters over de **seriële poort**.
- Een variabele type 'string' zoals in bijvoorbeeld C# bestaat niet echt.

C-taal – C-string

- Bekijk een voorbeeld waarbij je de tekst "Reboot\r\n" wil versturen via USART2.
- Zet een break point en bekijk het eindestringteken.
- Zoek ook de functie `fputc()` en bekijk hoe ze werkt...



```
94
95  /* USER CODE END SysInit */
96
97  /* Initialize all configured peripherals */
98  MX_GPIO_Init();
99  MX_ADC1_Init();
100 MX_USART2_UART_Init();
101 /* USER CODE BEGIN 2 */
102 // Laten weten dat we opgestart zijn, via de USART2 (USB).
103 strcpy(text, "Reboot\r\n");
104 printf("%s", text);
105 /* USER CODE END 2 */
106
107 /* Infinite loop */
108 /* USER CODE BEGIN WHILE */
109 while (1)
```

Watch 1

Name	Value	Type
text	0x200001B4 text[] "Reb...	uchar[101]
[0]	0x52 'R'	uchar
[1]	0x65 'e'	uchar
[2]	0x62 'b'	uchar
[3]	0x6F 'o'	uchar
[4]	0x6F 'o'	uchar
[5]	0x74 't'	uchar
[6]	0x0D	uchar
[7]	0x0A	uchar
[8]	0x00	uchar
[9]	0x00	uchar
[10]	0x00	uchar

Merk op dat we in de module 'Microcontrollers', een andere functie gebruikten. In de plaats van `printf()` hadden we toen `StringToUsart2()`...

```
46 void StringToUsart2(char* string)
47 {
48     uint8_t indexer = 0;
49
50     while(string[indexer] != 0)
51     {
52         // Byte versturen.
53         USART2->TDR = (uint8_t)string[indexer++];
54
55         // Wachten tot byte vertuurd is.
56         while((USART2->ISR & USART_ISR_TC) != USART_ISR_TC);
57     }
58 }
```

Bekijk het voorbeeld i.v.m. pointers en zoek uit hoe alles werkt...

```
33  /* Private typedef -----*/
34  /* USER CODE BEGIN PTD */
35  // Maak een struct die de toestand van het Nucleo Extension Shield bijhoudt.
36  // Die struct zou je later kunnen gebruiken om bijvoorbeeld in een C#-app
37  // real time de NES-status weer te geven...
38  typedef struct{
39      uint8_t leds;
40      uint8_t buttons;
41      uint16_t adValue;
42  }NES;
43  /* USER CODE END PTD */
```



```
65 // Maak een 8-bit unsigned integer.
66 static uint8_t count = 0;
67 // Maak een pointer (= plaats voor een verwijzing) naar een 8-bit unsigned integer.
68 static uint8_t* countPointer;
69
70 // Maak een struct aan van het type NES.
71 static NES nes;
72 // Maak een pointer (= plaats voor een verwijzing) naar een struct van het type NES.
73 static NES* nesPointer;
74
75 // Maak een array met plaats voor karakters. Merk op: als je enkel de naam van het array gebruikt,
76 // kan je die bekijken als een pointer naar het begin van dat array...
77 static char text[10];
78 // Maak een pointer naar een karakter.
79 static char* textPointer;
80 // Maak plaats voor één karakter.
81 static char character;
82
83 // Maak een pointer aan naar een void. Daarin kan je verwijzen naar eender welk type...
84 static void* voidPointer;
85 // Maak variabele aan die een AD-waarde kan bevatten.
86 static uint16_t voidPointerAdValue;
```

C-taal – pointers naar structs

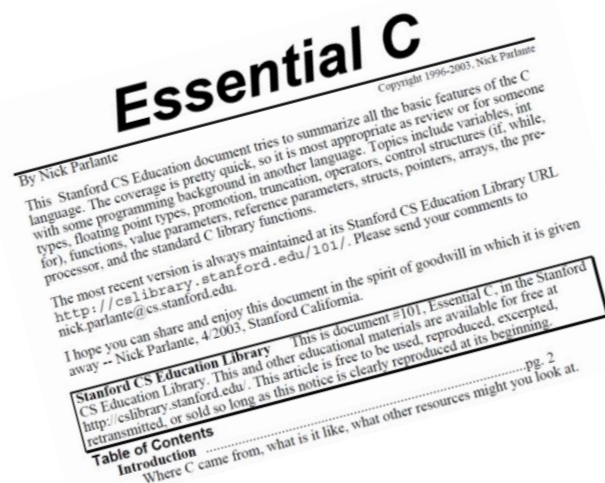
```
398 void UpdateNesState(NES* nes)
399 {
400     // De toestand van de LED's opvragen en opslaan in de struct.
401     nes->leds = 0;
402     if(HAL_GPIO_ReadPin(LED1_GPIO_Port, LED1_Pin) == GPIO_PIN_SET)
403         nes->leds |= 0x01;
404     if(HAL_GPIO_ReadPin(LED2_GPIO_Port, LED2_Pin) == GPIO_PIN_SET)
405         nes->leds |= 0x02;
406     if(HAL_GPIO_ReadPin(LED3_GPIO_Port, LED3_Pin) == GPIO_PIN_SET)
407         nes->leds |= 0x04;
408     if(HAL_GPIO_ReadPin(LED4_GPIO_Port, LED4_Pin) == GPIO_PIN_SET)
409         nes->leds |= 0x08;
410     if(HAL_GPIO_ReadPin(LED5_GPIO_Port, LED5_Pin) == GPIO_PIN_SET)
411         nes->leds |= 0x10;
412     if(HAL_GPIO_ReadPin(LED6_GPIO_Port, LED6_Pin) == GPIO_PIN_SET)
413         nes->leds |= 0x20;
414     if(HAL_GPIO_ReadPin(LED7_GPIO_Port, LED7_Pin) == GPIO_PIN_SET)
415         nes->leds |= 0x40;
416     if(HAL_GPIO_ReadPin(LED8_GPIO_Port, LED8_Pin) == GPIO_PIN_SET)
417         nes->leds |= 0x80;
```

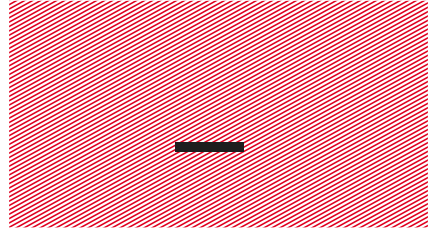
C-taal – void pointers

```
// Deel 4: pointer naar een void.  
// Je kan verwijzen naar gelijk wat als je een void pointer gebruikt...  
// Bekijk de inhoud van voidPointer na onderstaand statement. Je zal er het adres  
// van 'nes' in terugvinden.  
voidPointer = &nes;  
// Na correct casten, kan je ook de velden van de struct aanspreken.  
voidPointerAdValue = ((NES*)voidPointer)->adValue;
```

Het spreekt voor zich dat er nog veel meer te vertellen valt over de C-taal.

Wil je meer info, bekijk dan zeker de PDF op Toledo met als titel: **"Essential C"**. Geschreven door Nick Parlante. Echt **de moeite!**





Theorieopdracht





C-taal - oefeningen

Download het codevoorbeeld “Nucleo-L476RG – Pointers” en los de bijhorende opdrachten op.

Zie: **“Theorieopdracht 1 – C-taal”**.